

Stoquify Approval, Attestation, Evidence and Gate Control Plane — target architecture

G1 technical remediation, authentic approval requirements, and controlled next steps

BLOCKER AND GATE CONTROL REPORT

Prepared 2026-08-18

Evidence-bound | approval-gated | fail-closed

No production claim without authentic evidence.

Prepared: 2026-08-18 Decision status: proposed for governance and engineering review Initial pilot: POS G1 only

Architecture decision in one sentence

Extend Stoquify's existing Workflow Assurance modular-monolith capability with authority, artifact, policy, approval, evidence-envelope and independent-verification modules; keep domain truth in domain services, run scheduled work through existing worker/outbox patterns, and integrate external e-signature providers only through an optional adapter.

Design principles

- 1 **Automate facts, not authority or intent.** The system may collect evidence and route work; only authorized humans approve.
- 2 **Extend one control plane.** Workflow Assurance incidents, runs, alerts, waivers and control-tower UI remain canonical.
- 3 **Domain truth stays domain-owned.** The control plane consumes bounded evidence adapters; it does not mutate accounting, payroll, POS or inventory records directly.
- 4 **Bind every approval to exact content.** Artifact digest, policy version, decision, option and evidence-envelope digest are mandatory.
- 5 **Fail closed and explain why.** Missing, stale, ambiguous or conflicting evidence becomes a structured blocker.
- 6 **Preserve history.** Correction uses invalidation, supersession and new entries rather than destructive edits.
- 7 **Separate creation from verification.** The verifier recomputes evidence and enforces checker independence where policy requires it.
- 8 **Start narrow.** Implement G1 first, then generalize only after the pilot proves the contracts.

System context

Code or command excerpt

```
flowchart LR
  U[Authorized human approvers] --> UI[Stoquify approval and assurance UI]
  GOV[HR / security / governance owners] --> UI
  UI --> ACT[Protected server actions]
  ACT --> CP[Workflow Assurance control plane]
  CP --> AA[Authority registry]
  CP --> AR[Artifact registry]
  CP --> PC[Policy catalog]
  CP --> AW[Approval workflow]
  CP --> VE[Independent verifier]
  CP --> GE[Gate evaluator]
  D[Domain services] --> AD[Governed evidence adapters]
  AD --> CP
  CP --> BE[Business events and outbox]
  CP --> CT[Assurance control tower]
  CT -. optional .-> ES[External e-signature provider]
  GE --> RG[Release and domain gates]
```

The production module consumes governed records and service contracts. Repository scanning remains a developer/audit tool and is not a production source of authority.

Component boundaries

Code or command excerpt

```
flowchart TB
  subgraph Presentation
    Inbox[Approval inbox]
    Tower[Assurance control tower]
    AuthorityUI[Authority workbench]
    EvidenceUI[Evidence and verification viewer]
  end

  subgraph ProtectedActions
    ReadActions[Read actions]
    DecisionActions[Fresh-auth decision actions]
    AdminActions[Authority/policy admin actions]
  end

  subgraph AssuranceDomain
    services[services/assurance]
    Artifact[Artifact registry + canonical digests]
    Authority[Authority + delegation registry]
    Policy[Decision and gate policy]
  end
```

```

catalog] Approval[Approval orchestration] Envelope[Evidence envelopes]
Verify[Independent verification] Gate[Gate composition] Incident[Existing
runs/findings/incidents/waivers] end

subgraph PlatformServices Auth[Session assurance / fresh auth] RBAC[RBAC + tenant +
entitlement] Events[Business event + outbox] Audit[Audit log] Notify[Assurance alerts]
Storage[Controlled artifact storage] end

subgraph DomainAdapters POS[POS evidence adapter] Finance[Finance/close adapter]
Payroll[HRIS/payroll adapter] Inventory[Inventory/AP adapter]
Compliance[Country/compliance adapter] Release[Repository/release evidence intake] end

Presentation --> ProtectedActions ProtectedActions --> AssuranceDomain AssuranceDomain
--> PlatformServices DomainAdapters --> AssuranceDomain

```

Component specifications

1. Governed evidence connectors

Ownership: each domain owns its adapter; Assurance owns the adapter interface.

Responsibilities:

- 1 return organization-scoped, permission-safe evidence descriptors;
- 1 include source type, source ID, source version, observed time and source digest;
- 1 distinguish unavailable, stale, partial and conflicting evidence;
- 1 redact sensitive values before cross-domain projection;
- 1 never silently fall back to a different source;
- 1 expose no mutation of domain truth.

The repository evidence connector is a separate developer/release adapter. It may read registered manifests and CI evidence, not arbitrary production server files.

2. Artifact registry and canonical digest service

Ownership: Assurance platform, with domain owner for each artifact.

Responsibilities:

- 1 register artifact identity, tenant, domain, owner, URI and media type;
- 1 create immutable artifact versions;
- 1 assign a versioned canonicalization profile;
- 1 compute sha256 server-side and support independent recomputation;
- 1 freeze, supersede and retire artifacts;
- 1 emit drift/invalidation events;
- 1 reject approval requests for mutable or unverified artifacts.

Canonicalization profiles must be explicit. Binary files hash exact bytes. JSON uses a versioned canonical key/order/number/string policy. PDF/DOCX approvals bind the final exported bytes and any separately normalized semantic payload required by policy.

3. Authority and delegation registry

Ownership: HR/security/governance source owners; Assurance stores governed assignments and evidence references.

Responsibilities:

- 1 map stable `User . id` to governance authority codes;
- 1 record organization, scope, effective dates, appointment source and evidence digest;
- 1 store qualification type, jurisdiction and expiry when applicable;
- 1 support bounded delegation and revocation;
- 1 prevent authority from being derived from display names;
- 1 expose candidate, confirmed, expired, conflicted and revoked states;
- 1 preserve historical assignments after supersession.

An application role may be a candidate signal. It is never the sole authority source for a controlled decision.

4. Decision and gate policy catalog

Ownership: control owner plus domain owner; security/country review where touched.

Responsibilities:

- 1 version decision definitions, permitted options and required authority codes;
- 1 define quorum, sequence, fresh-auth maximum age and authentication level;
- 1 define same-person multi-role and maker-checker rules;
- 1 bind policy to artifact types and gate obligations;
- 1 define expiry, review, waiver and external-signature requirements;
- 1 publish only after an explicit policy-review workflow;
- 1 preserve the exact version used by historical approvals.

5. Approval orchestration

Ownership: Assurance service; decision content and consequences remain domain-owned.

Responsibilities:

- 1 expand a policy into unique approval obligations;
- 1 route requests only to active, eligible authorities;
- 1 display the canonical decision payload before action;
- 1 require explicit approve/reject intent and fresh authentication;
- 1 recompute the request digest server-side;
- 1 enforce tenant, entitlement, permission, authority, conflict and SoD rules;
- 1 write approval entry, authentication attestation, audit log and business event transactionally;
- 1 support rejection, expiry, cancellation, supersession and invalidation;
- 1 never edit an existing approval into a different decision.

6. Authentication-attestation adapter

Ownership: Security/authentication platform.

Responsibilities:

- 1 reuse the verified Stoquify session and `requireFreshAuth()`;
- 1 capture stable user ID, session ID reference, tenant, assurance method/level and verification time;

- 1 support stronger policy levels such as WebAuthn/passkeys when available;
- 1 store no password, TOTP secret, token or passkey private material;
- 1 reject session/tenant/identity mismatches;
- 1 allow authentication-method upgrades without changing approval semantics.

7. Immutable evidence envelope

Ownership: Assurance evidence service.

The canonical envelope contains:

- 1 schema and canonicalization versions;
- 1 organization and approval-request IDs;
- 1 policy ID/version/digest;
- 1 decision ID and selected option;
- 1 artifact ID/version/URI/digest;
- 1 approver stable ID and verified authority assignment ID;
- 1 authentication-attestation ID and timestamps;
- 1 rationale/evidence references subject to redaction;
- 1 approval-entry ID, status and server time;
- 1 prior/superseded record references when applicable.

The service hashes the canonical envelope and produces controlled JSON/PDF exports. PDF is a presentation; the canonical JSON digest is the machine-verification anchor. External signature artifacts remain linked evidence with their own digests.

8. Independent verification engine

Ownership: Controls assurance; independent checker role where policy requires it.

Responsibilities:

- 1 independently reload the artifact and policy versions;
- 1 recompute digests;
- 1 validate identity, tenant, authority scope/effectivity and qualification;
- 1 validate fresh authentication and approval timing;
- 1 validate quorum, role coverage, sequence and SoD;
- 1 detect revocation, expiry, supersession and drift;
- 1 create a deterministic verification record with reason codes;
- 1 invalidate prior verification when dependencies change;
- 1 require a separate human checker only for policies that demand human certification.

Automated cryptographic verification is not a human approval. Both may be required.

9. Gate composition and blocker engine

Ownership: Assurance platform; each gate has a named domain/control owner.

Responsibilities:

- 1 compose technical assurance checks, approval obligations and external evidence requirements;

- 1 evaluate an acyclic dependency graph;
- 1 preserve each input digest and policy version in GateRun;
- 1 use NOT_EVALUATED, not PASSED, when dependencies are blocked;
- 1 generate/update WorkflowAssuranceIncident blockers;
- 1 expose owner, safe detail and next permitted action;
- 1 invalidate downstream passes on material changes;
- 1 keep release authorization separate from domain-gate pass.

10. Operational read models and UI

Extend the existing Assurance Control Tower with:

- 1 **My approvals:** pending, expiring, rejected and completed requests.
- 1 **Gate overview:** dependencies, technical checks, approvals and external evidence.
- 1 **Blocker work queue:** owner, age, severity, evidence freshness and next action.
- 1 **Authority workbench:** assignments, delegations, qualifications, conflicts and expiry.
- 1 **Artifact registry:** versions, digests, drift and supersession.
- 1 **Verification queue:** automated results and human-checker work.
- 1 **Evidence viewer/export:** canonical payload, redactions, digest and verification result.

All states require text and icon cues, keyboard operation, focus management and EN/FR content. UI visibility is not authorization.

11. Notifications and escalation

Reuse WorkflowAssuranceAlertDelivery for in-app and webhook delivery. Add approval-specific templates, expiry reminders and escalation policies. Dedupe by obligation/version, record delivery state, and keep message content redacted. A delivered or opened notification cannot satisfy an obligation.

12. External e-signature adapter

Define a provider-neutral port:

- 1 create envelope from a frozen Stoquify decision payload;
- 1 receive signed/completed/cancelled webhooks idempotently;
- 1 verify provider signature and expected signer mapping;
- 1 store provider envelope ID, audit-certificate digest and final artifact digest;
- 1 attach evidence to, but do not replace, Stoquify authority and policy checks.

This adapter is optional for internal G1 attestations unless governance explicitly requires external signature evidence.

Target data model

Existing models should be reused where indicated. Names below are logical; final Prisma names require a separate reviewed schema design.

ControlDefinition	Map to/extend WorkflowAssuranceCheckDefinition	Global versioned key; optional tenant override reference	Published versions immutable	Required permission, owner, source contract, policy digest

GateDefinition	New	Versioned gate key	Immutable after publish	Dependency and obligation composition; no cycles
GateDependency	New	Unique gate version + dependency	Immutable with gate version	Acyclic graph validation
GateRun	New, linked to assurance runs	Tenant + gate/version + execution key	Append-only result; supersedable	Input manifest digest, idempotency, evaluator version
GateCheckResult	Map to assurance run/finding plus join	Tenant + gate run + obligation key	Append-only	Exact source/check-run reference
Blocker	Reuse WorkflowAssuranceIncident	Tenant + check/version/source identity	Lifecycle events; no destructive history	Source hash, owner, due date, redaction
Artifact	New	Tenant + artifact key	Metadata mutable; identity stable	Domain owner, type, access policy
ArtifactVersion	New	Tenant + artifact + version	Frozen versions immutable	URI, media type, state, canonicalization version
ArtifactDigest	New or child of version	Artifact version + algorithm + profile	Append-only	Server and independent-verification timestamps
AuthorityAssignment	New	Tenant + authority code + subject + scope + period	Active/revoked/superseded	Appointment evidence digest, stable user ID, qualification
AuthorityDelegation	Generalize HRIS pattern	Tenant + delegator/delegatee/scope/period	Active/revoked/superseded	Delegability, maker-checker, source digest
DecisionDefinition	New	Policy/version + decision ID	Immutable after publish	Options, artifact binding, expiry policy
DecisionRoleRequirement	New	Decision version + authority code + ordinal	Immutable	Quorum, sequence, SoD group, external-signature flag
ApprovalRequest	New	Tenant + policy/artifact/decision + idempotency key	Draft/ready/pending/final/in validated	Request digest, expiry, artifact/policy digests
ApprovalEntry	New	Tenant + request + requirement + approver; unique active obligation	Append-only; rejected/superseded/invalidated	Explicit intent, authority assignment, envelope digest
AuthenticationAttestation	New	Tenant + approval entry	Immutable snapshot	Method/level/time/session reference; no secret material
EvidenceEnvelope	New	Tenant + subject + schema version + digest	Immutable; supersedable exports	Canonical payload, redacted export, retention class
VerificationRun	New or specialized assurance run	Tenant + subject digest + verifier version + execution key	Append-only	Automated/human class, verifier ID, reason codes
ExceptionOrWaiver	Reuse/extend WorkflowAssuranceWaiver	Tenant + incident/policy	Requested/approved/rejected/expired/revoked	Requester != approver, expiry, evidence/policy digest
PolicyVersion	New	Policy key + version	Immutable after publish	Full policy digest and effective dates

ExternalSignatureEnvelope	New, optional	Tenant + provider + provider envelope ID	Provider lifecycle; history retained	Webhook idempotency, final artifact and audit digests
NotificationDelivery	Reuse WorkflowAssuranceAlertDelivery	Tenant + incident/obligation/channel/dedupe	Delivery lifecycle	Redaction, retry/dead-letter recovery

Every tenant-owned table requires `organizationId`, composite tenant-safe uniqueness, tenant-filtered indexes and `onDelete: Restrict` for evidence-bearing relationships. Historical evidence should use retention/supersession rather than cascade deletion.

Authority relationships

Code or command excerpt

```
erDiagram
    ORGANIZATION ||--o{ USER : contains
    USER }o--o{ APPLICATION_ROLE : receives_access
    USER ||--o{ AUTHORITY_ASSIGNMENT : may_hold
    AUTHORITY_ASSIGNMENT ||--o{ AUTHORITY_DEFINITION : grants
    AUTHORITY_ASSIGNMENT ||--o{ AUTHORITY_DELEGATION : may_delegate
    USER ||--o{ AUTHORITY_DELEGATION : delegates_or_receives
    AUTHORITY_ASSIGNMENT ||--o{ QUALIFICATION_EVIDENCE : supported_by
    DECISION_REQUIREMENT }o--|| AUTHORITY_DEFINITION : requires
    APPROVAL_ENTRY }o--|| AUTHORITY_ASSIGNMENT : exercised_under
```

Approval sequence

Code or command excerpt

```
sequenceDiagram
    participant O as Orchestrator
    participant A as Approver
    participant UI as Approval UI
    participant P as Protected action
    participant AR as Authority registry
    participant AS as Auth/session assurance
    participant DB as Transactional store
    participant E as Business event/outbox
```

```
O->>AR: Resolve eligible authority for obligation
O-->>A: Route pending request
A->>UI: Open exact decision and artifact
UI->>P: Request decision challenge
P->>AS: Require fresh authentication
AS-->>P: Verified user/tenant/method/time
A->>UI: Explicit Approve or Reject
UI->>P: Decision + request version + anti-replay token
P->>AR: Revalidate authority and SoD
P->>DB: Recompute payload/envelope digests
P->>DB: Write approval + attestation + audit atomically
P->>E: Record approval event/outbox in transaction
P-->>UI: Receipt with evidence-envelope digest
```

Evidence verification sequence

Code or command excerpt

```
sequenceDiagram
    participant V as Verifier
    participant R as Approval store
    participant AR as Artifact registry
    participant PR as Policy registry
    participant AU as Authority registry
    participant G as Gate evaluator
```

```
V->>R: Load immutable approval envelope
V->>AR: Reload artifact bytes and recompute digest
V->>PR: Reload policy version and recompute digest
V->>AU: Validate authority, qualification and effectivity
V->>R: Validate authentication, timing, quorum and SoD
V->>R: Persist deterministic verification result
alt all obligations verified
V->>G: Publish verified obligation set
else any failure
V->>G: Publish reason codes and blocker
end
evidence end
```

Gate-evaluation sequence

Code or command excerpt

```
sequenceDiagram
    participant C as Gate coordinator
    participant D as Dependency resolver
    participant T as Technical assurance
    participant A as Approval verifier
    participant X as External evidence adapters
    participant I as Incident service
```

```
C->>D: Resolve versioned dependency graph
alt dependency blocked
D-->>C: NOT_EVALUATED + dependency blockers
else dependencies ready
C->>T: Load verified technical check runs
C->>A: Load verified approval obligations
C->>X: Load verified external evidence
C->>C: Hash input manifest and evaluate
fail closed
alt all required inputs pass
C-->>I: Resolve/supersede prior blockers
else missing or failed input
C-->>I: Create/update structured blockers
end
end
```

Gate dependency flow

Code or command excerpt

```
flowchart LR
    G1[G1 architecture/control freeze] --> G2[G2 transaction/access trust]
    G2 --> G3A[G3A provider payment truth]
    G2 --> G3B[G3B receipt/fiscal truth]
    G2 --> G4[G4 workstation candidate]
    G3B --> G4
    G3A --> G5[G5 store financial close]
    G3B --> G5
    G4 --> G5
    G5 --> G6[G6 physical fulfillment]
    G3A --> G7[G7 order-to-cash and returns]
    G3B --> G7
    G6 --> G7
    G3A --> G8[G8 operational readiness]
    G3B --> G8
    G4 --> G8
    G5 --> G8
    G6 --> G8
    G7 --> G8
    G8 --> G9[G9 bounded pilot/expansion]
```

Invalidation flow

Code or command excerpt

```
flowchart TD
    Change{Material change detected} -->|artifact bytes| AD[New artifact digest]
    Change -->|policy version| PD[New policy digest]
    Change -->|authority revoked/expired| AU[Authority invalid]
    Change -->|authentication/evidence defect| EV[Evidence invalid]
    AD --> IA[Invalidate affected approvals]
    PD --> IA
    AU --> IA
    EV --> IA
    IA --> IV[Invalidate verification results]
    IV --> IG[Invalidate affected gate runs]
    IG --> DG[Mark downstream gates not evaluated]
    DG --> BI[Open/update blockers and notify owners]
```

Invalidation never deletes history. It writes a reasoned invalidation record and creates a new path to reapproval.

Lifecycle state machines

Approval request

Code or command excerpt

```
stateDiagram-v2
    [*] --> DRAFT
    DRAFT --> READY: policy + artifact frozen
    READY --> PENDING: obligations routed
    PENDING --> PARTIALLY_APPROVED: some valid approvals
    PARTIALLY_APPROVED --> APPROVED: quorum/roles complete
    APPROVED --> VERIFIED: independent verification passes
    PENDING --> REJECTED
    PARTIALLY_APPROVED --> REJECTED
    READY --> CANCELLED
    PENDING --> EXPIRED
    PARTIALLY_APPROVED --> EXPIRED
    APPROVED --> INVALIDATED: drift/revocation/verification failure
    VERIFIED --> INVALIDATED: dependency changes
    DRAFT --> SUPERSEDED
    READY --> SUPERSEDED
```

Artifact

Code or command excerpt

```
stateDiagram-v2 [*] --> DRAFT DRAFT --> FROZEN: canonical digest recorded FROZEN -->
APPROVED: policy-defined approvals verified FROZEN --> SUPERSEDED: new version
registered APPROVED --> SUPERSEDED: material change/new version SUPERSEDED --> RETIRED
```

Gate

Code or command excerpt

```
stateDiagram-v2 [*] --> NOT_EVALUATED NOT_EVALUATED --> COLLECTING_EVIDENCE:
dependencies eligible COLLECTING_EVIDENCE --> BLOCKED: missing/failed input
COLLECTING_EVIDENCE --> READY: obligations complete READY --> EVALUATING EVALUATING -->
PASSED EVALUATING --> FAILED PASSED --> NOT_EVALUATED: dependency invalidated FAILED -->
COLLECTING_EVIDENCE: remediation starts BLOCKED --> COLLECTING_EVIDENCE: source changes
```

Exception/waiver

Code or command excerpt

```
stateDiagram-v2 [*] --> REQUESTED REQUESTED --> UNDER_REVIEW UNDER_REVIEW --> APPROVED
UNDER_REVIEW --> REJECTED APPROVED --> EXPIRED APPROVED --> REVOKED
```

No state transition may be performed only in the browser. Each transition is a server transaction with actor, policy, source digest, idempotency and audit evidence.

Proposed service and action contracts

Suggested bounded modules:

Code or command excerpt

```
services/assurance/artifacts/* services/assurance/authority/*
services/assurance/policies/* services/assurance/approvals/*
services/assurance/evidence-envelopes/* services/assurance/verification/*
services/assurance/gates/* services/assurance/adapters/*

actions/assurance/artifacts.actions.ts actions/assurance/authority.actions.ts
actions/assurance/approvals.actions.ts actions/assurance/verification.actions.ts
actions/assurance/gates.actions.ts
```

Representative commands:

```
1 registerArtifactVersion
1 freezeArtifactVersion
1 assignAuthority
1 delegateAuthority
1 revokeAuthority
1 publishDecisionPolicy
1 createApprovalRequest
1 recordApprovalDecision
1 verifyApprovalRequest
1 runGateEvaluation
1 invalidateControlEvidence
1 exportEvidenceEnvelope
```

Every mutation derives `organizationId` and actor identity from the trusted session. Inputs may contain resource IDs but never trusted tenant or approver identity.

Business events

Minimum event catalog:

<code>control.artifact.frozen</code>	Artifact digest and version become approval-eligible.
<code>control.artifact.superseded</code>	Triggers dependent invalidation.
<code>control.authority.assigned</code>	Records governed appointment.
<code>control.authority.revoked</code>	Invalidates future use and policy-defined active approvals.
<code>control.approval.requested</code>	Obligations become routable.
<code>control.approval.decided</code>	Records explicit approve/reject intent.
<code>control.approval.invalidated</code>	Preserves reason and prior envelope reference.
<code>control.evidence.envelope.created</code>	Makes canonical evidence digest available.
<code>control.verification.completed</code>	Publishes pass/fail reason codes.
<code>control.gate.evaluated</code>	Records gate input manifest and result.
<code>control.gate.invalidated</code>	Propagates dependency invalidation.

Each event uses the existing organization-scoped idempotency and payload-hash pattern.

G1 pilot design

Policy expansion

The G1 policy imports D-01 through D-11 from the frozen contract and expands each `requiredApproverRoles` array into one obligation per role. The resulting key is deterministic:

Code or command excerpt

```
G1:0.2.0:<contract-sha256>:<decision-id>:<authority-code>
```

The 11 decisions must produce exactly 33 obligations. A mismatch is a policy-import blocker, not an approval shortage.

Complete approval example

Code or command excerpt

```
{ "schemaVersion": "1.0.0", "organizationId": "org-example", "approvalRequestId":
  "g1-d01-request", "decisionId": "D-01", "selectedOption":
  "DISABLE_HIDE_AND_REJECT_STORE_CREDIT", "artifactId":
  "STOQUIFY-POS-G1-CONTRACT-FREEZE-0.2.0-20260817", "artifactVersion": "0.2.0",
  "artifactSha256": "11434eb3e1af1826516426e90d2d53a2faa47ade361d91191b5f3e0c950a36db",
  "policyVersion": 1, "authorityCode": "PRODUCT_OWNER", "approverUserId":
  "verified-user-id", "authorityAssignmentId": "confirmed-authority-assignment",
  "decision": "APPROVED", "freshAuthenticatedAt": "2026-08-18T14:00:00.000Z",
  "approvedAt": "2026-08-18T14:02:00.000Z", "evidenceEnvelopeSha256":
```

```
"example-generated-by-server" }
```

This example is structural only and grants no approval credit because the example identities and envelope digest are not live evidence.

Required failure examples

Incomplete D-07	Product and finance present; qualified Cameroon reviewer missing	Request remains <code>PARTIALLY_APPROVED</code> ; G1 blocker remains open.
Authority conflict	Candidate has application role but no active appointment, or two sources disagree	No approval request routed as authoritative; open <code>AUTHORITY_CONFLICT</code> .
Expired authentication	Approval occurs outside policy freshness window	Reject transaction with <code>FRESH_AUTH_REQUIRED</code> ; create no approval entry.
Artifact drift	Recomputed contract digest differs from frozen digest	Invalidate affected approvals and verification; G1 returns to blocked.
SoD violation	Policy forbids same user satisfying two incompatible roles or verifier equals producer	Reject obligation/verification with <code>SEGREGATION_OF_DUTIES_VIOLATION</code> .
Existing handwriting image	Media exists without trusted envelope/audit evidence	Preserve as attachment with zero approval credit.

Gate handoff

The current G1 script should eventually consume a generated, read-only verified approval manifest rather than accept hand-edited approval JSON. During transition:

- 1 keep the current contract and fail-closed script;
- 2 generate a proposed register from runtime records;
- 3 independently compare it with the contract requirements;
- 4 make the script trust only `VERIFIED` envelopes and an exact manifest digest; and
- 5 retain a non-writing report mode for development and CI.

Platform applicability

POS releases	Gate dependencies, approvals, artifacts, evidence and blockers	Sale, tender, drawer and POS session rules
Destructive migrations	Candidate manifest, hashes, maker/checker, restore evidence	Migration execution and database history
Country packs	Qualification, provenance, review and expiry	Statutory rules and regulator sources
Payments/treasury	Payment-release authority and evidence envelopes	Provider state, settlement and bank truth
Reconciliation	Sign-off/quorum and certificate evidence	Matching, suspense and exception logic
Accounting close	Approval/waiver routing and exports	Ledger, periods, close calculations and certification
Inventory/write-offs	Authority, materiality policy and maker-checker	Stock movements, valuation and disposition
Purchasing/AP	Approval obligations and evidence	PO, receipt, invoice, supplier and payment workflow
Payroll/compensation	Authority, inbox and evidence	Employee, contract, compensation and payroll calculations
Module activation	Policy, approval and audit	Entitlement and provisioning state
Compliance	Evidence envelope and qualified review	Fiscal documents, submissions and authority responses
AI agents	Human approval boundary and activation evidence	Agent runtime, tool policy and evaluation results
Production release/rollback	Composite gate, artifacts and release decisions	CI/CD provider and infrastructure deployment truth

Security and abuse analysis

Cross-tenant approval	Derive tenant from session; composite tenant keys; tenant-scoped lookups; negative tests.
Forged or client-edited approval payload	Reconstruct canonical payload server-side and compare version/anti-replay token.
Stale or fabricated authority	Appointment evidence, source digest, effective dates, revocation and conflict state.
UI bypass/direct API call	All controls in service/protected action; UI only presents state.
Replay or duplicate approval	Unique obligation key, idempotency key and request-digest conflict detection.
Artifact substitution	Registered immutable version, canonicalization profile and independent digest recomputation.
Requester/self-approver abuse	Policy-driven SoD, maker-checker and deny-wins rules.
Producer self-verification	Separate verifier capability/role and policy-enforced actor inequality.
Audit/evidence mutation	Append-oriented tables, restricted update paths, supersession events, backup/restore and optional hash-chain anchoring after review.
Secret or personal-data exposure	Store references and assurance metadata, not credentials; redact UI, logs, alerts and exports.
Clock manipulation	Server time, monitored clock synchronization and invalid future timestamps.
AI agent impersonation	Mark service/agent principals; prohibit them from human approval and risk-acceptance capabilities.
External provider spoofing	Signed webhook verification, allowlisted provider adapter, idempotency and final artifact/audit digest verification.

Observability and recovery

Metrics:

- 1 pending obligations by gate/role and age;
- 1 authority assignments nearing expiry;
- 1 approval freshness failures;
- 1 verification failures by reason code;
- 1 artifact/policy drift invalidations;
- 1 gate evaluation duration and blocked time;
- 1 stuck runs, alerts and outbox messages;
- 1 dead-letter recovery outcomes;
- 1 cross-tenant and SoD denial counts.

Runbooks must cover:

- 1 authority source unavailable;
- 1 artifact storage unavailable;
- 1 verifier backlog;
- 1 partial transaction/outbox recovery;
- 1 corrupted or mismatched evidence export;
- 1 mistaken authority assignment;

- 1 artifact supersession after approval;
- 1 external-signature provider outage;
- 1 restore and independent evidence reconciliation.

Deployment recommendation

Code or command excerpt

```
flowchart LR
    subgraph CurrentStoquify [Existing Stoquify deployment]
        Web[Next.js web/actions]
        Assurance[Assurance services]
        Worker[Scheduler/outbox worker]
        DB[(PostgreSQL)]
        Obj[(Controlled artifact storage)]
    end
    Web --> Assurance
    Assurance --> DB
    Assurance --> Obj
    Worker --> Assurance
    Worker --> DB
    Assurance -. provider-neutral adapter .-> ESign[External e-sign provider]
    Assurance -. telemetry .-> Obs[Logs/metrics/alerts]
```

Start inside the existing deployment. Preserve internal module interfaces so artifact hashing, verification or external-signature integration can be extracted later if scale, independent administration, cryptographic key isolation or availability objectives justify it.

Extraction triggers should be evidence-based:

- 1 materially different availability/SLO from the application;
- 1 independent operations/security administration requirement;
- 1 verification throughput that harms transactional workloads;
- 1 regulated key-management isolation;
- 1 multiple products consuming the service across deployment boundaries.

Until then, a separate microservice adds network failure, distributed transaction and operational costs without solving the primary governance gap.

Architecture acceptance conditions

The target architecture is ready for Phase 0/1 implementation only when:

- 1 governance approves the authority-source and delegation model;
- 1 security approves authentication-assurance levels and sensitive permissions;
- 1 data governance approves retention, redaction and export classes;
- 1 G1 domain owners approve the imported decision/role policy;
- 1 qualified-review requirements remain explicit and unfilled rather than guessed;
- 1 the additive schema migration and rollback design are independently reviewed; and
- 1 no implementation claims G1 approval or legal-signature validity by construction.